Phala Network Substrate 블록체인 노드 분석

1. Phala Node 개요

Phala Node는 **Substrate 프레임워크 기반의 블록체인 노드**로, Phala Network의 핵심 블록체인 인프라를 제공합니다. TEE 컴퓨팅과 통합된 독특한 블록체인 아키텍처를 구현합니다.

2. 노드 구조 분석

2.1 핵심 의존성 구조

```
# Substrate 핵심 컴포넌트
├── sc-service: 노드 서비스 프레임워크
├── sc-client-api: 클라이언트 API
├── sc-consensus-babe: BABE 합의 알고리즘
├── sc-consensus-grandpa: GRANDPA 최종성 가젯
├── sc-network: P2P 네트워킹
├── sc-rpc: RPC 서버
├── sc-telemetry: 텔레메트리 시스템

# Phala 특화 컴포넌트
├── phala-node-rpc-ext: Phala RPC 확장
├── phala-pallets: Phala 팰릿 모음
```

```
├── node-runtime: Phala 런타임
├── node-rpc: 커스텀 RPC 구현
└── node-executor: 런타임 실행기
```

2.2 바이너리 구성

```
// 메인 바이너리 구조 (추정)
[[bin]]
name = "phala-node"
path = "bin/main.rs"
required-features = ["cli"]

// 라이브러리 타입
[lib]
crate-type = ["cdylib", "rlib"]
```

3. 합의 메커니즘

3.1 BABE (Blind Assignment for Blockchain Extension)

```
// BABE 설정 예시 (추정)
struct BabeConfig {
// 슬롯 지속시간 (6초)
slot_duration: Duration::from_secs(6),
```

```
// 에포크 길이 (2400 슬롯 = 4시간)
epoch_length: 2400,
// VRF 키 생성
authorities: Vec<BabeAuthorityId>,
// 랜덤성 소스
randomness: [u8; 32],
}


impl BabeBlockImport {
fn  verify_slot(&self, header: &Header) -> Result<(), ConsensusError> {
// 1. 슬롯 번호 검증
let  slot = self.extract_slot(header)?;
// 2. VRF 증명 검증
let  vrf_proof = self.extract_vrf_proof(header)?;
self.verify_vrf_proof(&vrf_proof, slot)?;
// 3. 권한 검증
let  authority = self.get_slot_authority(slot)?;
self.verify_authority_signature(header, &authority)?;
Ok(())
}
}
```

3.2 GRANDPA (GHOST-based Recursive ANcestor Deriving Prefix Agreement)

```
// GRANDPA 설정 예시 (추정)
struct GrandpaConfig {
// 검증자 세트
authorities: Vec<GrandpaAuthorityId>,
// 라운드 지속시간
round_duration: Duration::from_secs(60),
// 투표 임계값 (2/3 + 1)
threshold: u64,
}


impl GrandpaVoter {
fn  cast_vote(&mut  self, round: u64, target: BlockHash) -> Result<Vote, VotingE
// 1. 라운드 검증
self.validate_round(round)?;
// 2. 대상 블록 검증
self.validate_target_block(&target)?;
// 3. 투표 생성
let  vote = Vote {
round,
target_hash: target,
target_number: self.get_block_number(&target)?,
};
```

```
// 4. 투표 서명
let signed_vote = self.sign_vote(&vote)?;
// 5. 네트워크에 브로드캐스트
self.broadcast_vote(&signed_vote)?;
Ok(vote)
}
}
```

4. 네트워킹 시스템

4.1 P2P 네트워크 구성

```
// 네트워크 설정 예시 (추정)
struct NetworkConfig {
// 노드 식별자
node_key: NodeKey,
// 리스닝 주소
listen_addresses: Vec<Multiaddr>,
// 부트스트랩 노드
boot_nodes: Vec<MultiaddrWithPeerId>,
// 최대 피어 수
max_peers: u32,
// 프로토콜 버전
```

```
protocol_version: ProtocolVersion,

}


impl NetworkService {

async  fn  handle_peer_connection(&mut  self, peer_id: PeerId) -> Result<(), Net

// 1. 피어 인증

self.authenticate_peer(&peer_id).await?;

// 2. 프로토콜 협상

let  protocols = self.negotiate_protocols(&peer_id).await?;

// 3. 동기화 시작

self.start_sync_with_peer(&peer_id, &protocols).await?;

// 4. 피어 상태 추적

self.track_peer_state(&peer_id);

Ok(())

}

}
```

4.2 블록 동기화

```
// 블록 동기화 구현 (추정)

struct SyncManager {

// 동기화 상태

sync_state: SyncState,
```

```
// 피어 정보
peers: HashMap<PeerId, PeerInfo>,
// 요청 큐
request_queue: VecDeque<BlockRequest>,
}

impl SyncManager {
async fn sync_blocks(&mut self) -> Result<(), SyncError> {
loop {
match self.sync_state {
SyncState::Idle => {
// 동기화 필요성 확인
if self.needs_sync().await? {
self.sync_state = SyncState::Downloading;
}
}
SyncState::Downloading => {
// 블록 다운로드
let blocks = self.download_blocks().await?;
// 블록 검증 및 임포트
for block in blocks {
self.import_block(block).await?;
}
```

```
// 동기화 완료 확인
if  self.is_sync_complete().await? {
self.sync_state = SyncState::Idle;
}
}
}
tokio::time::sleep(Duration::from_millis(100)).await;
}
}
}
```

5. RPC 시스템

5.1 표준 RPC API

```
// RPC 서버 설정 (추정)
struct RpcConfig {
// HTTP 서버 설정
http_addr: SocketAddr,
http_cors: Option<Vec<String>>,
// WebSocket 서버 설정
ws_addr: SocketAddr,
ws_max_connections: u32,
```

```
// 허용된 메서드
allowed_methods: Vec<String>,
}


impl RpcServer {
fn  setup_rpc_methods(&mut  self) -> Result<(), RpcError> {
// 1. 시스템 RPC
self.register_method("system_name", |_| Ok("Phala Node".to_string()))?;
self.register_method("system_version", |_| Ok(env!("CARGO_PKG_VERSION").to_strin
// 2. 체인 RPC
self.register_method("chain_getBlock", |params| self.get_block(params))?;
self.register_method("chain_getBlockHash", |params| self.get_block_hash(params))
// 3. 상태 RPC
self.register_method("state_getStorage", |params| self.get_storage(params))?;
self.register_method("state_call", |params| self.call_runtime(params))?;
// 4. Phala 특화 RPC
self.register_phala_rpc_methods()?;
Ok(())
}
}
```

5.2 Phala 특화 RPC 확장

```
// Phala RPC 확장 (phala-node-rpc-ext)
trait PhalaRpcApi {
// TEE 워커 정보 조회
fn  get_worker_info(&self, worker_id: WorkerId) -> Result<WorkerInfo, RpcError>;
// 컨트랙트 상태 조회
fn  get_contract_state(&self, contract_id: ContractId) -> Result<ContractState,
// Pink 런타임 쿼리
fn  pink_query(&self, contract_id: ContractId, input: Vec<u8>) -> Result<Vec<u8>
// 클러스터 정보 조회
fn  get_cluster_info(&self, cluster_id: ClusterId) -> Result<ClusterInfo, RpcErr
}


impl PhalaRpcApi for PhalaRpcImpl {
fn  get_worker_info(&self, worker_id: WorkerId) -> Result<WorkerInfo, RpcError>
// 1. 워커 ID 검증
self.validate_worker_id(&worker_id)?;
// 2. 온체인 상태에서 워커 정보 조회
let  worker_info = self.client
.runtime_api()
.get_worker_info(&BlockId::Hash(self.client.info().best_hash), worker_id)
.map_err(|e| RpcError::RuntimeError(e.to_string()))?;
// 3. 추가 정보 수집 (오프체인)
```

```
let  extended_info = self.collect_extended_worker_info(&worker_id)?;

Ok(WorkerInfo {

basic_info: worker_info,

extended_info,

})

}

}
```

6. 텔레메트리 시스템

6.1 메트릭 수집

```
// 텔레메트리 설정 (추정)

struct TelemetryConfig {

// 텔레메트리 엔드포인트

endpoints: Vec<TelemetryEndpoint>,

// 수집 간격

collection_interval: Duration,

// 메트릭 필터

metric_filters: Vec<MetricFilter>,

}


impl TelemetryCollector {
```

```
fn collect_metrics(&mut self) -> TelemetryData {
TelemetryData {
// 노드 정보
node_info: self.collect_node_info(),
// 네트워크 메트릭
network_metrics: self.collect_network_metrics(),
// 블록체인 메트릭
blockchain_metrics: self.collect_blockchain_metrics(),
// 시스템 메트릭
system_metrics: self.collect_system_metrics(),
// Phala 특화 메트릭
phala_metrics: self.collect_phala_metrics(),
}
}
fn collect_phala_metrics(&self) -> PhalaMetrics {
PhalaMetrics {
// TEE 워커 수
active_workers: self.count_active_workers(),
// 컨트랙트 실행 수
contract_executions: self.count_contract_executions(),
// Pink 런타임 상태
pink_runtime_status: self.get_pink_runtime_status(),
// 클러스터 상태
```

```
cluster_status: self.get_cluster_status(),
}
}
}
```

6.2 성능 모니터링

```
// 성능 모니터링 구현 (추정)
struct PerformanceMonitor {
// 블록 처리 시간
block_processing_times: VecDeque<Duration>,
// 트랜잭션 처리량
transaction_throughput: MovingAverage,
// 메모리 사용량
memory_usage: MemoryTracker,
// CPU 사용률
cpu_usage: CpuTracker,
}

impl PerformanceMonitor {
fn  track_block_import(&mut  self, block: &Block, import_time: Duration) {
// 1. 블록 처리 시간 기록
self.block_processing_times.push_back(import_time);
```

```
if  self.block_processing_times.len() > 1000 {
self.block_processing_times.pop_front();
}
// 2. 트랜잭션 처리량 업데이트
let  tx_count = block.extrinsics().len() as f64;
self.transaction_throughput.add_sample(tx_count / import_time.as_secs_f64());
// 3. 성능 경고 확인
if  import_time > Duration::from_secs(10) {
log::warn!("Slow block import detected: {:?}", import_time);
}
// 4. 메트릭 업데이트
self.update_performance_metrics();
}
}
```

7. 클라이언트 API

7.1 블록체인 클라이언트

```
// 클라이언트 구현 (추정)
struct PhalaClient {
// Substrate 클라이언트
client: Arc<Client>,
```

```
// 트랜잭션 풀
transaction_pool: Arc<TransactionPool>,
// 네트워크 서비스
network: Arc<NetworkService>,
}


impl PhalaClient {
async fn submit_extrinsic(&self, extrinsic: Extrinsic) -> Result<TxHash, Clien
// 1. 트랜잭션 검증
self.validate_extrinsic(&extrinsic).await?;
// 2. 트랜잭션 풀에 추가
let tx_hash = self.transaction_pool
.submit_one(&BlockId::Hash(self.client.info().best_hash), extrinsic)
.await?;
// 3. 네트워크에 전파
self.network.propagate_transaction(&tx_hash).await?;
Ok(tx_hash)
}
async fn get_block(&self, block_hash: BlockHash) -> Result<Option<Block>, Clie
// 1. 로컬 스토리지에서 조회
if let Some(block) = self.client.block(&BlockId::Hash(block_hash))? {
return Ok(Some(block.block));
}
```

```
// 2. 네트워크에서 요청
let block = self.network
.request_block(block_hash)
.await?;
Ok(block)
}
}
```

7.2 상태 쿼리 API

```
// 상태 쿼리 구현 (추정)
impl StateApi for PhalaClient {
fn query_storage(&self, key: &StorageKey, at: Option<BlockHash>) -> Result<Opti
let block_id = match at {
Some(hash) => BlockId::Hash(hash),
None => BlockId::Hash(self.client.info().best_hash),
};
self.client
.storage(&block_id, key)
.map_err(|e| StateError::ClientError(e.to_string()))
}
fn call_runtime_api<R: Decode>(&self, method: &str, args: &[u8], at: Option<Blo
let block_id = match at {
```

```
Some(hash) => BlockId::Hash(hash),

None => BlockId::Hash(self.client.info().best_hash),

};

let  result = self.client

.runtime_api()

.call_api_at(&block_id, method, args)

.map_err(|e| StateError::RuntimeError(e.to_string()))?;

R::decode(&mut &result[..])

.map_err(|e| StateError::DecodeError(e.to_string()))

}

}
```

8. 개발 및 테스트 도구

8.1 CLI 인터페이스

```
// CLI 명령어 구조 (추정)

#[derive(clap::Parser)]

struct PhalaNodeCli {

#[clap(subcommand)]

subcommand: Option<Subcommand>,

#[clap(flatten)]

run: sc_cli::RunCmd,
```

```
}


#[derive(clap::Subcommand)]
enum Subcommand {
/// 키 관리 명령어
Key(sc_cli::KeySubcommand),
/// 체인스펙 생성
BuildSpec(sc_cli::BuildSpecCmd),
/// 체인스펙 확인
CheckBlock(sc_cli::CheckBlockCmd),
/// 데이터베이스 내보내기
ExportBlocks(sc_cli::ExportBlocksCmd),
/// 벤치마크 실행
Benchmark(frame_benchmarking_cli::BenchmarkCmd),
/// Try-runtime 명령어
TryRuntime(try_runtime_cli::TryRuntimeCmd),
}
```

8.2 벤치마킹 시스템

```
// 벤치마크 구현 (추정)
impl BenchmarkRunner {
fn  run_runtime_benchmarks(&self, pallet: &str, extrinsic: &str) -> Result<Bench
```

```
// 1. 벤치마크 환경 설정
let mut externalities = self.create_benchmark_externalities()?;
// 2. 벤치마크 실행
let result = externalities.execute_with(|| {
// 팰릿별 벤치마크 실행
match pallet {
"phala" => self.benchmark_phala_pallet(extrinsic),
"system" => self.benchmark_system_pallet(extrinsic),
_ => Err(BenchmarkError::UnknownPallet(pallet.to_string())),
}
})?;
// 3. 결과 분석
let analysis = self.analyze_benchmark_result(&result)?;
Ok(BenchmarkResult {
execution_time: result.execution_time,
memory_usage: result.memory_usage,
weight: analysis.calculated_weight,
analysis,
})
}
}
```

이 문서는 Phala Network의 Substrate 블록체인 노드 구현을 다룹니다. 다음 문서에서는 런타임 구현을 상세히 분석합니다.